

Design approach and logic

Hyperparams

10000 epochs: 2000 was not enough, although around 6500-7000 the memory got saturated and it unlearned.

Larger memory size: more stable training and more options so that it explores a larger state space.

Gamma 0.98: more greedy, otherwise it likes to build too much.

LR 1e-3: tried lower learning rates, but didn't learn much anymore.

Epsilon decay 0.999: otherwise randomness is gone too fast and it doesn't learn anything anymore

Grad clip norm (removed): first time I tried for overnight training, I asked gpt to tell me what I should do and it said to use it to ensure that single huge gradients don't break the training. Back then it didn't train enough, but now, I think it could've been useful.

Resume epsilon 0.5: when resuming training, I want the epsilon to resume at 0.5 to still provide randomness challenge, but it shouldn't unlearn everything.

Ideas

Could've tried adding epsilon reset, so that it resets epsilon every N episodes to something like 0.5.

Additionally, it would lower learning rate and clip gradients more to balance it out, so it doesn't unlearn everything but still get's put into challenging situations.

Features

Features to describe columns:

```
fill_heights: normalized column heights (highest piece / len(rows))
```

```
diff_heights: normalized diff heights centered around the mean (can go negative and positive)
```

```
total_holeyness: fraction of holes of total height (count of holes / total_height / board_area)
```

```
height_deviation: std of fill_heights
```

```
lowest_point: min fill_heights
```

```
highest_point: max fill_heights
```

```
mean was skipped since from shuffling the features, it showed low impact (probably covered by min and max).
```

features to describe holes in columns:

```
hole_height_per_col: hole counts per col normalized to heights (holeyness per column)
```

```
vertical_hole_depths: normalized mean hole depths per col
```

```
vertical_hole_clusterdness: 1 - std of hole heights per col
```

features to describe holes in rows:

```
horizontal_hole_distances: mean distance of holes from center per row
```

global features:

```
mean_hole_depth: mean of hole depths
```

```
mean_hole_vertical_clusterdness: mean of vertical hole clusterdness
```

```
mean_hole_edge_distance: mean of hole edge distance
```

```
next_piece: one-hot-vector indicating the class of the next piece (therefore actually a feature per tetrimonoe type)
```

Column features are so it understands the current surface it is playing with.

Hole features so it understands how holes are mapped and how it can optimize play for future. I guess hole horizontal distance could've been kept as left to right or right to left instead so it describes the space more, but at the same time, columns describe this info as well, so maybe it was redundant overall?

Global features just to summarize the board and maybe help make a better scoring of the state.

Next piece so it can learn to avoid losing the game. I guess I could've made a special case where if it is an obvious loss (cannot place next) and it chooses it, then it gets penalized already.

Model

Final model combined normal linear layers with convolutional layers.

fill_heights, diff_heights, hole_height_per_col, vertical_hole_depths, vertical_hole_clusterdness were stacked into a 5 channel input, which was input into the convolutional encoder. The idea was, that these features describe the state surface and holes and help the model decide on the value better. Instead of ReLU, SiLU is used to not completely kill off negative gradients. After dense layers, there is layernorm, which helps normalize gradients and learning.

horizontal_hole_distances were fed into a single dense row encoder, since the data was very minimal compared to the previous block.

The global info was put into a single dense summary encoder.

Next piece was put into a next piece encoder, with the goal to have the encoder learn what each piece represents and better map to the understanding.

Finally, all 4 encoder outputs were put together and fed through a fully connected head layer, which outputs the final score.

This architecture splits the different features up into separate parts hopefully helping the model differentiate between valuable and non valuable info.

At first the model was a lot heavier but it wouldn't learn because of its' size, so it was slimmed down to the current result, which managed to gain a pretty good score (averages 200+ and sometimes gets 500+ lines, from training plot actually high score of nearly 1200 lines. Unfortunately, the checkpoint was done a bit before the highest performing model, after which the memory saturated and it stopped learning).

For checkpoint 3, I submit the unlearned, ep 7000, model, meanwhile, for the 4th, I submit the best, ep 6500, model.